

Almacenando datos. Ficheros

Esteban Álvarez

Entrada y salida

- Un programa es un **proceso**
- El **esquema general** de cualquier proceso es:

Entrada → **proceso** → **Salida**

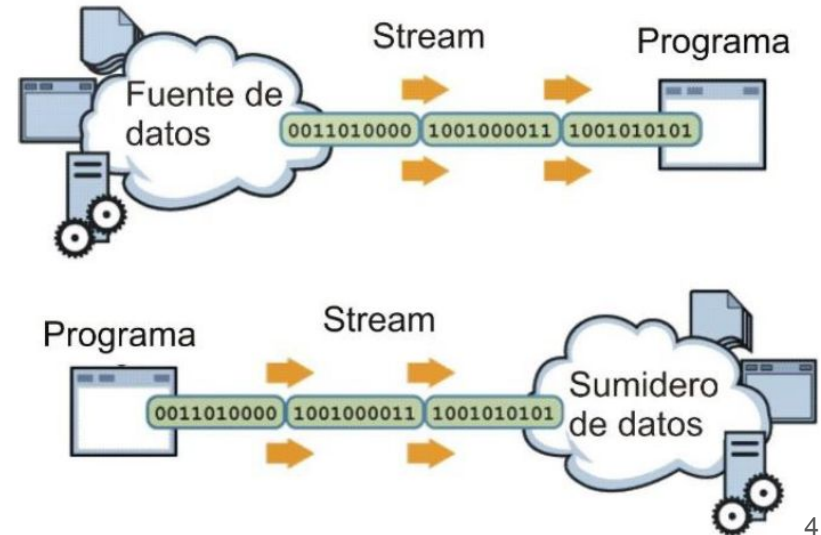
- Esta entrada de datos la hacíamos a través de **variables temporales** que se almacenan en la RAM
- Pero a veces es necesario **almacenar** esta información para volver a utilizarla.
- Para eso utilizaremos **ficheros**, almacenando los datos de forma persistente o permanente

Entrada y salida

- Dos tipos de Entrada/Salida (E/S)
 - **E/S estándar.** La utilizada hasta ahora: se introducen datos a través de teclado y se muestra información por consola
 - **E/S a través de ficheros.** Podemos apoyarnos en ficheros tanto para leer información como para almacenarla
- Todas las operaciones de E/S se encuentran en la librería **java.io**

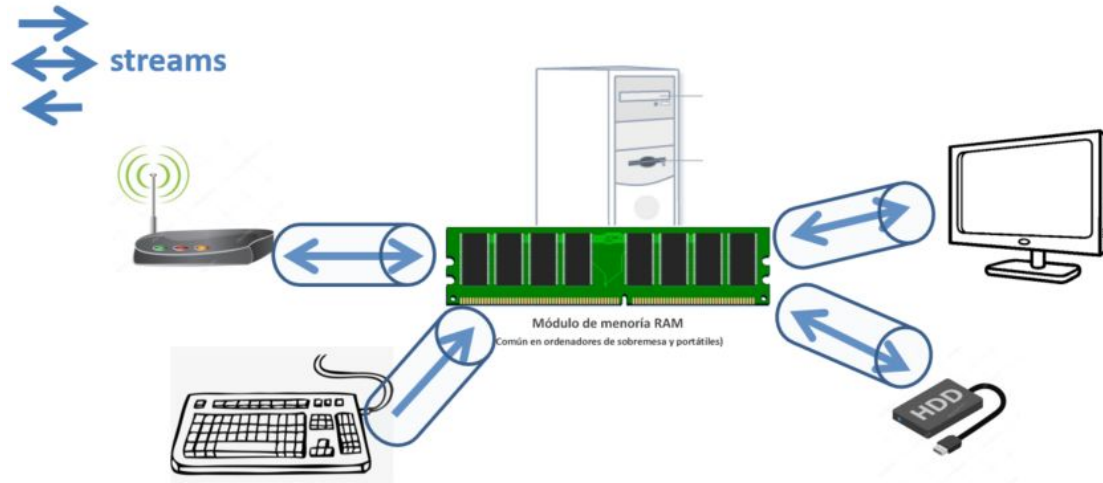
Control de flujo y clases relativas a flujos

- Un **flujo o stream** es algo que produce o consume información
- La clase Stream representa un flujo de datos, es decir, un conjunto secuencial de bytes, como puede ser un archivo, un dispositivo de entrada/salida, memoria, un conector TCP/IP, etc.
- **Cualquier programa** realizado en Java que necesite llevar a cabo una operación de **entrada/salida** lo hará a través de un **Stream**.



Control de flujo y clases relativas a flujos

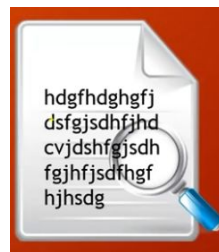
- La entrada/salida de Java se organiza generalmente mediante **objetos llamados Streams**
- Crearemos **objetos Stream** que asociaremos a los dispositivos de E/S
- Toda la información que **entre** en la aplicación (a través de la RAM) o **salga** de ella lo hace como stream



Control de flujo y clases relativas a flujos

Clasificación de los streams en función de los datos que transportan:

- **Flujos de bytes.** Para leer/escribir información en binario. Típicamente son ficheros con la extensión .dat que no se pueden abrir o editar directamente. Otros ejemplos son: gif, png, jpg, bin...
- **Flujos de caracteres.** Para leer/escribir caracteres en ficheros de texto tipo .txt o [.CSV](#)



Control de flujo y clases relativas a flujos

¿Cuándo usar uno u otro?



- **Flujos de bytes.** 8. bits. Suele ser información que se comparte entre programas o se envía a través de la red.
No es necesario que la información se pueda leer por un humano en forma de texto: ficheros de configuración, cálculos que se almacenan para posteriores ejecuciones de la aplicación, objetos...



- **Flujos de caracteres.** 16 bits. Queremos que la información se escriba en formato texto

Control de flujo y clases relativas a flujos

EJERCICIO

¿Flujo de bytes o de caracteres? ¿Entrada o salida?

1. Aplicación que lee datos de un fichero (que ha creado el usuario) para crear objetos y añadirlos a una colección. ¿Qué extensión tendría ese fichero?
2. Aplicación que al finalizar la ejecución guarda en un fichero todos los datos de los objetos que ha ido añadiendo el usuario a una colección para recuperarlos y seguir trabajando con esos mismos datos en la siguiente ejecución. ¿Qué extensión tendría ese fichero?
3. Aplicación que tiene una opción de “exportar” y escribe datos en un fichero para luego poder hacer tablas y gráficos en Excel. ¿Qué extensión tendría ese fichero?

Control de flujo y clases relativas a flujos

Clases en Java que hay que utilizar



- Flujos de bytes
 - InputStream para leer de un fichero binario
 - OutputStream para escribir en él



- Flujos de caracteres
 - Reader
 - Writer

Control de flujo y clases relativas a flujos

Las clases anteriores son interfaces y en realidad se trabajará con sus implementaciones:



- Flujos de bytes

- InputStream/OutputStream

- **FileInputStream**: permite leer bytes de un fichero.
 - **FileOutputStream**: permite escribir bytes en un fichero o descriptor.
 - **BufferedInputStream**: permite leer datos a través de un flujo con un buffer intermedio.
 - **BufferedOutputStream**: implementa los métodos para escribir en un flujo a través de un buffer.

Control de flujo y clases relativas a flujos

Las clases anteriores son interfaces y en realidad se trabajará con sus implementaciones:



- Flujos de caracteres

- Reader/Writer

- **StreamReader**: es un flujo de caracteres cuya fuente es una cadena de caracteres o string.
 - **StreamWriter**: es un flujo de caracteres cuya salida es un buffer de cadena de caracteres, que puede utilizarse para construir un string.

Flujos

Para **utilizar un flujo**, las **acciones** que hay que realizar son:

Lectura

1. Abrir un flujo de una fuente de datos (creación del stream)
 - Teclado
 - **Fichero**
 - Socket local
2. Mientras existan datos disponibles:
 - Leer datos del fichero
3. Cerrar el flujo (método close)

Flujos

Para **utilizar un flujo**, las **acciones** que hay que realizar son:

Escritura

1. Abrir un flujo a un destino de datos (creación del stream)
 - Pantalla
 - **Fichero**
 - Socket local
2. Mientras existan datos disponibles:
 - Escribir datos en el fichero
3. Cerrar el flujo (método close)

Flujos. Clase File

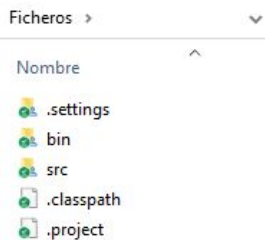
- Antes de trabajar con flujos propiamente dichos, ya sean flujos de bytes o de caracteres, es recomendable crear un objeto **File**
- Consiste en “**convertir**” un **fichero** físico ubicado en nuestro PC en un **objeto java**:

```
File archivo1 = new File("un_path/un_fichero.txt");  
File archivo2 = new File("C:\\Usuarios\\XXXX\\Mis  
                        Documentos\\MiArchivo.txt");  
File archivo3 = new File("/MiAnimacion.gif"); // Carpeta Raiz  
  
archivo1.createNewFile(); //Crea físicamente el fichero. No utilizar
```

- Fíjate que hay que usar doble barra en las rutas absolutas para “escapar” el caracter \

Flujos. Clase File

- Si no especificamos una ruta, se toma como ruta relativa el **directorio de trabajo** (directorio del proyecto en la carpeta del workspace)



```
File archivo4 = new File ("datos.txt");
```

- Con File también podemos trabajar con **directorios**:

```
//Accedemos al directorio en la carpeta de trabajo  
File directorio = new File("Carpeta Nueva");  
//Se crea el directorio físicamente  
directorio.mkdir();
```

Flujos. Clase File

Cuando hacemos: `File archivo4 = new File ("datos.txt");` el fichero “datos.txt” puede existir o no:

- Si **no existe** y
 - queremos leer tenemos un problema
 - queremos escribir se crea el fichero y se escribe en él
- Si **existe** y
 - queremos leer no hay ningún problema (es lo deseable)
 - queremos escribir se sobrescribe (elimina) la información anterior. Aunque este comportamiento se puede modificar para añadir en lugar de sobrescribir

Flujos. Clase File

El objetivo final de usar un objeto File es “localizar” o “enlazar” el fichero con el que vamos a trabajar y este puede existir o no

Flujos. Clase File

Ejercicios. Constructores de la clase file.

- 1.Crea un Objeto File asociado al fichero “personas.dat” que se encuentra en el directorio de trabajo

Flujos. Clase File

Ejercicios. Constructores de la clase file.

1.Crea un Objeto File asociado al fichero “personas.dat” que se encuentra en el directorio de trabajo

```
File f = new File("personas.dat");
```

Flujos. Clase File

Ejercicios. Constructores de la clase file.

2.Crea un Objeto File asociado al fichero personas.dat que se encuentra en el directorio “ficheros” dentro del directorio de trabajo

- Constructor con un string
- Constructor con dos strings
- Constructor con un file y un string

Flujos. Clase File

Ejercicios. Constructores de la clase file.

2.Crea un Objeto File asociado al fichero personas.dat que se encuentra en el directorio “ficheros” dentro del directorio de trabajo

- Constructor con un string
`File f = new File("ficheros/personas.dat");`
- Constructor con dos strings
`File f = new File("ficheros", "personas.dat");`
- Constructor con un file y un string
`File directorio=new File("ficheros");`
`File f=new File(directorio,"personas.dat");`

Flujos. Clase File

Ejercicios. Constructores de la clase file.

3. Crea un Objeto File asociado al fichero personas.dat dando la ruta absoluta
C:\ficheros\personas.dat

- Constructor con un string
- Constructor con dos strings
- Constructor con un file y un string

Flujos. Clase File

Ejercicios. Constructores de la clase file.

3. Crea un Objeto File asociado al fichero personas.dat dando la ruta absoluta C:\ficheros\personas.dat. Se puede usar la ruta desde C: o desde la raíz /

- Constructor con un string

```
File f = new File("c:\\ficheros\\personas.dat");
```

```
File f = new File("/ficheros/personas.dat");
```

- Constructor con dos strings

```
File f = new File("/ficheros", "personas.dat");
```

- Constructor con un file y un string

```
File directorio=new File("/ficheros");
```

```
File f=new File(directorio,"personas.dat");
```

Flujos. Clase File

Métodos de la clase File

- Analiza los métodos de la clase File
- **CONCLUSIÓN:** Es **recomendable** declarar el fichero a utilizar como File antes de pasárselo al Stream
- Nos permite hacer **comprobaciones**, como por ejemplo si existe el fichero, o la carpeta antes de abrir el flujo

Flujos. Clase File

Ejercicio. Utiliza el método `exists()` para comprobar si el fichero “nombres.txt” existe o no en el directorio de trabajo. Mostrar un mensaje por consola

Flujos. Clase File

Ejercicio. Utiliza el método exists() para comprobar si el fichero “nombres.txt” existe o no en el directorio de trabajo. Mostrar un mensaje por consola

```
File archivo = new File("nombres.txt");
if (archivo.exists()) {
    System.out.println("El fichero existe");
    System.out.println("Abrimos el flujo y empezamos a trabajar");
}else {
    System.out.println("Error. El fichero no existe");
}
```

Flujos. Clase File

Ejercicio. Modifica el ejercicio anterior para saber si existe la unidad “E:” (porque más adelante se va a crear un fichero en la unidad “E”)

Flujos. Clase File

Ejercicio. Modifica el ejercicio anterior para saber si existe la unidad “E:” (porque más adelante se va a crear un fichero en la unidad “E”)

```
File directorio = new File("e:");

if (directorio.exists()) {
    System.out.println("El directorio existe");
    System.out.println("Abrimos el flujo y empezamos a trabajar");
}else {
    System.out.println("Error. El directorio no existe");
}
```

Flujos de caracteres. Ficheros de texto

Para trabajar con ficheros de texto utilizaremos las clases de la librería [java.io](#):

- Leer
 - Reader
 - InputStreamReader
 - FileReader
 - BufferedReader
- Escribir
 - Writer
 - OutputStreamWriter
 - FileWriter
 - BufferedWriter
 - PrintWriter

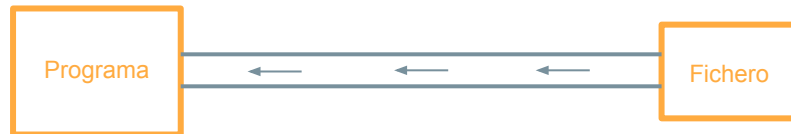
Flujos de caracteres. Ficheros de texto. Lectura

Para trabajar con ficheros de texto utilizaremos las clases de la librería [java.io](#):

- Leer
 - Reader
 - InputStreamReader
 - **FileReader.**
 - Lee carácter a carácter el fichero de entrada
 - Nos interesará sobre todo el método `read()`
 - **BufferedReader.**
 - Es la mejor forma de leer un fichero
 - Utiliza un buffer (memoria intermedia) para almacenar el fichero.
 - Permite leer líneas completas
 - Métodos `read()` y `readLine()`

Flujos de caracteres. Ficheros de texto. Lectura

Clase FileReader. Pasos:



1. Crear un **objeto File** con el fichero
2. Crear el **objeto FileReader**. Es el flujo de lectura propiamente dicho
3. **Leer** el fichero Utilizando el FileReader anterior. Sólo se permite leer un carácter cada vez
4. **Cerrar** el flujo

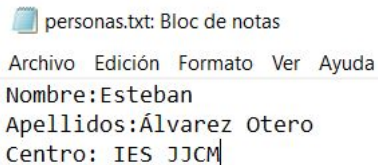
Flujos de caracteres. Ficheros de texto. Lectura

Clase FileReader

1. Crear un **objeto file** con el fichero existente

```
//personas.txt es un fichero que se encuentra en el  
//directorio de trabajo
```

```
File fichero = new File ("personas.txt");
```



2. Crear el **objeto FileReader**. Este es el flujo propiamente dicho

```
FileReader fr=new FileReader(fichero);
```

```
//También se podría omitir el objeto File con:
```

```
FileReader fr=new FileReader("personas.txt");
```

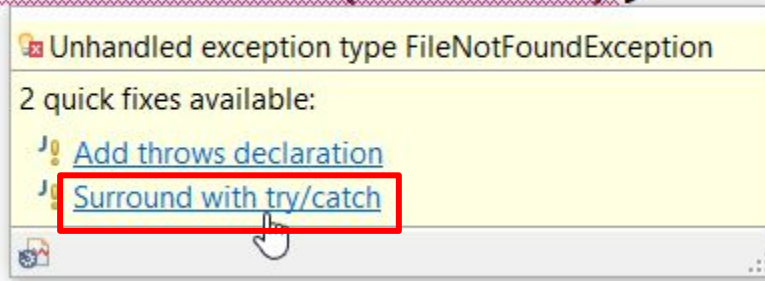

Flujos de caracteres. Ficheros de texto. Lectura

Clase FileReader

2. Crear el **objeto FileReader**. Este es el flujo propiamente dicho
 - a. El **constructor** de FileReader muestra un **error**
 - b. Nos obliga a capturar una **excepción** con un bloque **try/catch**

Presentación sobre excepciones

```
FileReader fr=new FileReader(fichero);
```



Flujos de caracteres. Ficheros de texto. Lectura

Clase FileReader

2. Crear el **objeto FileReader**. Este es el flujo propiamente dicho
 - a. El **constructor** de FileReader muestra un **error**
 - b. Nos obliga a capturar una **excepción** con un bloque **try/catch**
 - c. Mostrar al usuario un mensaje apropiado con el error

```
try {  
    FileReader fr=new FileReader(fichero);  
} catch (FileNotFoundException e) {  
    // TODO Auto-generated catch block  
    System.out.println("Error abriendo el fichero");  
}
```

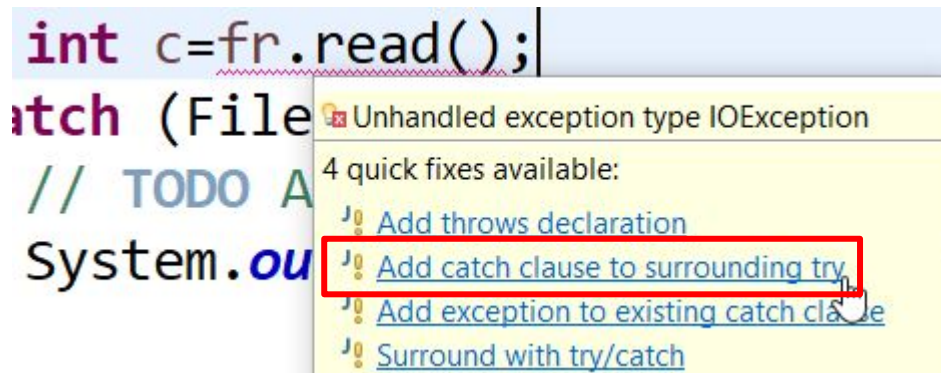
Flujos de caracteres. Ficheros de texto. Lectura

Clase FileReader

3. Leer el fichero utilizando el flujo anterior. `read()` devuelve un entero con el carácter leído del fichero o -1 si se ha llegado al final del fichero

```
int c=fr.read();
```

- También obliga a capturar una **excepción**. Capturar añadiendo un nuevo **catch**



Flujos de caracteres. Ficheros de texto. Lectura

Clase FileReader

3. Leer el fichero

```
int c=fr.read();
```

- Mostrar el carácter leído por pantalla y seguir leyendo

```
//-1 indica el fin del fichero
```

```
while(c!=-1) {
```

```
    //Cast a char para que muestre el caracter. Usamos print
```

```
    System.out.print((char)c);
```

```
    c=fr.read();
```

```
}
```

Flujos de caracteres. Ficheros de texto. Lectura

Clase FileReader

4. Cerrar el flujo
 - a. Una vez que se ha leído el fichero se cierra el flujo (a continuación del while)

```
fr.close();
```

Flujos de caracteres. Ficheros de texto. Lectura

Clase FileReader

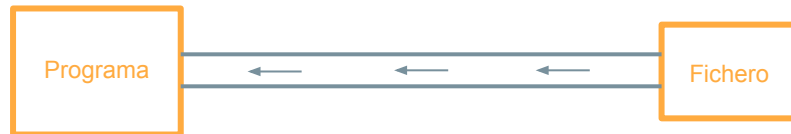
- Se puede añadir un **if** (`fichero.exists()`) antes de la lectura del fichero para hacer la comprobación

Flujos de caracteres. Ficheros de texto. Lectura

```
//personas.txt es un fichero que se encuentra en el
//directorio de trabajo
File fichero = new File ("personas.txt");
if (fichero.exists()) {
    try {
        FileReader fr = new FileReader(fichero);
        //Leemos del fichero con read(). Devuelve un entero
        int c = fr.read();
        //-1 indica el fin del fichero
        while (c != -1) {
            //Cast a char para que muestre el caracter
            System.out.print((char) c);
            c = fr.read();
        }
        fr.close();
    } catch (FileNotFoundException e) {
        System.out.println("Error abriendo el fichero");
    } catch (IOException e) {
        System.out.println("Error al leer");
    }
} else {
    System.out.println("El fichero no existe");
}
```

Flujos de caracteres. Ficheros de texto. Lectura

Clase FileReader. Pasos:



1. Crear un **objeto File** con el fichero

```
File fichero = new File ("fichero.txt");
```

2. Crear el **objeto FileReader**. Es el flujo de lectura propiamente dicho

```
FileReader fr = new FileReader(fichero);
```

3. **Leer** el fichero Utilizando el FileReader anterior. Sólo se permite leer un carácter cada vez

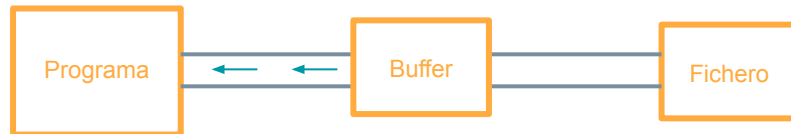
```
int c = fr.read();
```

4. **Cerrar** el flujo

```
fr.close();
```


Flujos de caracteres. Ficheros de texto. Lectura

Clase BufferedReader. Pasos:



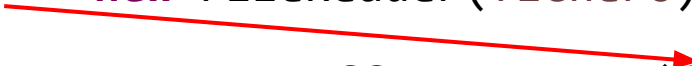
1. Crear un **objeto File** con el fichero
2. Crear el **objeto FileReader**. Es el flujo de lectura propiamente dicho
3. Crear el objeto **BufferedReader** partiendo del FileReader anterior
4. **Leer** el fichero El método `readline()` del `BufferedReader` anterior, permite leer una línea completa
5. **Cerrar** el flujo

Flujos de caracteres. Ficheros de texto. Lectura

Clase BufferedReader.

- El código de creación del objeto BufferedReader queda:

```
File fichero = new File ("personas.txt"); //Fichero  
FileReader fr = new FileReader(fichero); //Flujo  
BufferedReader br= new BufferedReader(fr); //Buffer
```



Flujos de caracteres. Ficheros de texto. Lectura

Clase `BufferedReader`.

- El código de creación del objeto `BufferedReader`. Más óptimo:

```
File fichero = new File ("personas.txt");
```

```
BufferedReader br= new BufferedReader(new FileReader(fichero));
```

Se recomienda usar esta forma para leer/escribir con Buffer porque al hacer `br.close()` se cierra tanto el buffer como el flujo.

Flujos de caracteres. Ficheros de texto. Lectura

Clase `BufferedReader`.

- Sólo queda **leer** cada línea del fichero
- `readLine()` lee una línea del fichero en formato `String`
- El fin del fichero se detecta cuando la cadena es vacía (**null**)

```
while((cadena=br.readLine())!=null) {  
    System.out.println(cadena);  
}
```

Flujos de caracteres. Ficheros de texto. Lectura

Clase BufferedReader.

```
try {  
    String cadena;  
    File fichero = new File ("personas.txt");  
    BufferedReader br= new BufferedReader(new FileReader(fichero));  
    while((cadena=br.readLine())!=null) {  
        System.out.println(cadena);  
    }  
    br.close();  
} catch (IOException e) {  
    e.printStackTrace();  
}
```

Flujos de caracteres. Ficheros de texto. Lectura

Cierre del flujo de datos

- Lo más conveniente para cerrar el flujo de datos es en una sentencia **finally**
- También hay que tratar la excepción...

```
finally {  
    try {  
        br.close();  
    } catch (Exception e) {  
        //No se suele poner código. Si hay problemas al cerrar el flujo es porque no se ha abierto.  
    }  
}
```

Flujos de caracteres. Ficheros de texto. Lectura

IMPORTANTE: Para la lectura de ficheros de texto se recomienda utilizar la clase `BufferedReader`

Flujos de caracteres. Ficheros de texto. Lectura

EJERCICIO

- Utilizando un editor de texto plano, crea un fichero de texto con varias líneas
- Lee el fichero línea a línea con un programa java utilizando un buffer stream
- Muestra cada línea por pantalla
- Muestra el número de líneas totales del fichero
- Indica si el número de líneas es par

- Muestra ahora solo las líneas impares

Flujos de caracteres. Ficheros de texto. Lectura

EJERCICIO

- Crea un fichero de texto con varias líneas. En cada línea solo hay un número entero
- Lee el fichero línea a línea con un programa java utilizando un buffer stream.
- Almacena los datos leídos del fichero en un ArrayList de enteros. Tratar la excepción de conversión de datos
- Muestra el contenido del ArrayList por pantalla
- Muestra cuántos números almacena el ArrayList

Flujos de caracteres. Ficheros de texto. Escritura

Para trabajar con ficheros de texto utilizaremos las clases de la librería [java.io](#):

- Escribir
 - Writer
 - OutputStreamWriter
 - **FileWriter**
 - Escribe caracter a caracter sobre el fichero
 - **BufferedWriter**
 - Escribe a través de un buffer
 - Método write()
 - **PrintWriter**
 - Escribe en formato texto **variables y objetos java**

Flujos de caracteres. Ficheros de texto. Escritura

Los pasos son los mismos que con la lectura. Escritura con **buffer**:

1. Crear un **objeto File** con el fichero
2. Crear el **objeto FileWriter**
3. Crear el objeto **BufferedWriter** partiendo del FileWriter anterior
4. **Escribir** en el fichero
5. **Cerrar** el flujo. **IMPORTANTÍSIMO** Si no se cierra el flujo, los datos no se guardan en el fichero

Flujos de caracteres. Ficheros de texto. Escritura

1. Crear un **objeto File** con el fichero

```
File fichero = new File ("personas.txt");
```

2. Crear el **objeto FileWriter**

- a. Si el fichero no existe en el disco duro, se crea
- b. Podemos especificar si queremos **sobreescribir** el fichero existente
 - i. Es la opción por defecto del constructor de FileWriter
- c. O **añadir** el texto al final del documento (**append**)
 - i. true para añadir

3. Crear el objeto **BufferedWriter** partiendo del FileWriter anterior

```
BufferedWriter bw= new BufferedWriter(new FileWriter(fichero, true));
```



Flujos de caracteres. Ficheros de texto. Escritura

4. **Escribir** en el fichero

```
//Se pueden escribir caracteres de escape como \n para nueva línea  
bw.write("\nEscribiendo con buffer");  
//O utilizar el método newLine()  
bw.newLine();  
bw.write("Otra línea");
```

5. **Cerrar** el flujo

```
bw.close();
```

Flujos de caracteres. Ficheros de texto. Escritura

Escritura con **buffer**. Código completo:

```
File fichero = new File ("personas.txt");  
BufferedWriter bw= new BufferedWriter(new FileWriter(fichero));  
bw.write("Escribiendo con buffer");  
bw.close();
```

Flujos de caracteres. Ficheros de texto. Escritura

Escritura con **PrintWriter**

- Se recomienda el uso de **Buffer** o **PrintWriter**
- La declaración es igual que la del Buffer
 - Objeto File
 - Objeto FileWriter. Sobre escribir por defecto. Opcional: append (añadir)
 - Objeto PrintWriter
- Su uso es muy similar a la salida por pantalla del System.out

```
File fichero = new File ("personas.txt");  
PrintWriter pw = new PrintWriter(new FileWriter(fichero));  
pw.println("PrintWriter");  
pw.close();
```

Flujos de caracteres. Ficheros de texto

EJERCICIO

Crea una aplicación con un menú de opciones para realizar las siguientes acciones:

1. Pedir 5 palabras o frases al usuario. Guarda cada palabra o frase en una línea nueva de un fichero de texto
2. Pedir palabras al usuario hasta que este introduzca la palabra “fin”
 - Afina el programa para que el usuario pueda escribir “fin” de cualquier forma: fin,FIN,FiN,FIn...
3. Muestra el fichero de texto
4. Busca una palabra en el fichero. Indica al usuario si existe o no la palabra buscada

Flujos de caracteres. Ficheros de texto

EJERCICIO

- Crea en tu escritorio un fichero de texto de nombre “ejercicio.txt” Edítalo con el bloc de notas y añade varias líneas en él
- Haz un nuevo proyecto java
- El Main debe leer todo el texto de “ejercicio.txt” y escribirlo en consola
- Modifica el ejercicio anterior para copiar línea a línea el texto de “ejercicio.txt” en “ejercicio_copia.txt”
- Utiliza Buffer para la lectura y PrintWriter para la escritura

Flujos de bytes. Ficheros binarios

- Cualquier fichero es susceptible de interpretarse como una secuencia de bytes

Clases a utilizar:

- Leer
 - `InputStream`
 - **`FileInputStream`**: permite leer bytes de un fichero
 - **`BufferedInputStream`**: permite leer datos a través de un flujo con un buffer intermedio
- Escribir
 - `OutputStream`
 - **`FileOutputStream`**: permite escribir bytes en un fichero
 - **`BufferedOutputStream`**: implementa los métodos para escribir en un flujo a través de un buffer.

Flujos de bytes. Ficheros binarios. Input/OutputStream

- Tanto para leer como para escribir en un flujo de bytes realizaremos las **mismas tareas** que para los ficheros de texto:
 1. Objeto File
 2. Objeto FileInputStream o FileOutputStream
 3. Leer/escribir
 - a. Leer. Método read()
 - b. Escribir. Método write()
 4. Cerrar el flujo
- También se podría utilizar un buffer para leer o escribir

Flujos de bytes. Ficheros binarios. Input/OutputStream

EJERCICIO

- Crea un proyecto nuevo
- En el Main escribe números enteros en un fichero binario de nombre “valores.dat” hasta que se introduzca el valor 0
- A continuación lee el fichero y muestra por consola los datos que contiene

Flujos de bytes. Ficheros binarios. Input/OutputStream

CLASES FILTRO (DataInputStream y DataOutputStream)

- Permiten **leer/escribir directamente datos** de tipo char, byte, short, int, long, float, double, boolean, String
- Hay que recordar que cada tipo de datos **ocupa** una cantidad de bytes diferente:
 - int 32 bits, long 64 bits...
- Por lo que hay que tener en cuenta el **orden** en el que se escribieron los datos
 - Si escribimos: int, float, int, int, String
 - Hay que leer en el mismo orden: int, float, int, int, String

Flujos de bytes. Ficheros binarios. Input/OutputStream

CLASES FILTRO (DataInputStream y DataOutputStream)

Uso:

1. File
2. FileInputStream
3. DataInputStream
4. Uso: readInt(), readShort(), readFloat()...

[Queda como ampliación](#)

Almacenamiento de objetos en ficheros. Serialización

- La **serialización** permite almacenar **objetos completos** en un fichero
- Consiste en **convertir** los distintos objetos de la memoria a **bytes** que se guardarán en un fichero.
- Se denomina **persistencia** a la posibilidad de almacenar los objetos en un fichero, de forma que estos persistan después de que la aplicación ha finalizado.
- Las clases **ObjectInputStream** y **ObjectOutputStream** están diseñadas para crear flujos de entrada y salida de objetos persistentes

Almacenamiento de objetos en ficheros. Serialización

- Proceso:
 - Serialización objeto → serializable → byte → grabar al fichero
 - Deserialización fichero → leer bytes → serializable → objeto
- La clase cuyos objetos van a persistir debe de implementar el interface **Serializable** del paquete java.io.

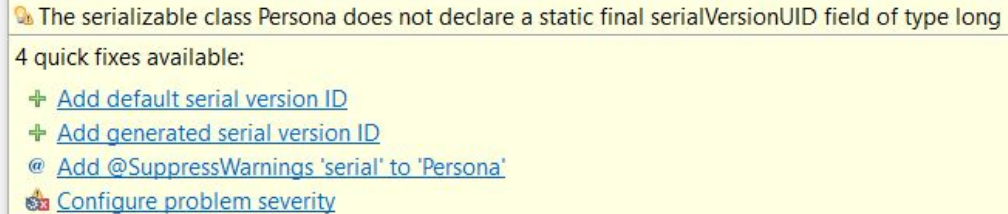
```
class Persona implements Serializable{
```

- En caso contrario, los objetos no se almacenan en el fichero

Si una clase es **componente** de otra, también debe ser serializable. Ejemplo:
Para guardar un Coche con un Motor, las dos clases deben ser serializables

Almacenamiento de objetos en ficheros. Serialización

```
public class Persona implements Serializable{
```



- Al implementar Serializable aparece un warning indicando que es recomendable añadir la constante `serialVersionUID`
- Se utiliza para que al leer el objeto del fichero, se asegure que es un objeto de esa clase Persona la que se lee y no otra
- No es obligatorio declarar el `serialVersionUID`

Almacenamiento de objetos en ficheros. Serialización

GRABAR OBJETOS EN FICHERO. ObjectOutputStream

1. Declarar File
`File f=new File("objetos.dat");`
2. Declarar FileOutputStream
`FileOutputStream fos=new FileOutputStream(f);`
3. Declarar ObjectOutputStream
`ObjectOutputStream oos=new ObjectOutputStream(fos);`
4. Guardar fichero con writeObject()
`Persona p=new Persona("Pepe", 25);`
`oos.writeObject(p);`
5. Cerrar el flujo
`oos.close();`

Almacenamiento de objetos en ficheros. Serialización

LEER OBJETOS DE UN FICHERO. ObjectInputStream

1. Declarar File

```
File f=new File("objetos.dat");
```

2. Declarar FileInputStream

```
FileInputStream fis=new FileInputStream(f);
```

3. Declarar ObjectInputStream

```
ObjectInputStream ois=new ObjectInputStream(fis);
```

4. Leer del fichero con readObject(). Devuelve **null** cuando ha llegado al final del fichero. Hay que hacer un cast al leer

```
Persona p=(Persona)ois.readObject();
```

5. Cerrar el flujo

```
ois.close();
```

Almacenamiento de objetos en ficheros. Serialización

LEER OBJETOS DE UN FICHERO. ObjectInputStream

- Para leer **varios objetos** de un fichero se puede utilizar un bucle **while** como en los otros ficheros

```
while((p=(Persona)ois.readObject())!=null) {  
    System.out.println(p.toString());  
}
```

- El **problema** es que `p=(Persona)ois.readObject()` genera una excepción al ejecutarse cuando llega al final del fichero
- Hay que tratar la excepción **EOFException**

Almacenamiento de objetos en ficheros. Serialización

EJERCICIO

1. Crea la clase Persona con los atributos Nombre y Edad, además de sus métodos y constructores
2. Almacena objetos de la clase Persona en un fichero. Añade al fichero tantas personas como desee el usuario (Utiliza un bucle while para hacerlo)
3. Lee el fichero anterior y almacena los objetos Persona en un ArrayList
4. Muestra el nombre de todas las personas almacenadas en el fichero utilizando el ArrayList anterior
5. Guarda el ArrayList **completo** (y no objetos persona) en otro fichero
6. Ampliación: Crea un menú de opciones para el ejercicio anterior

RESUMEN

RESUMEN

LECTURA DE FICHEROS DE TEXTO SIN BUFFER (CARÁCTER A CARÁCTER)

1. Crear un **objeto File** con el fichero

```
File fichero = new File ("fichero.txt");
```

2. Crear el **objeto FileReader**

```
FileReader fr = new FileReader(fichero);
```

3. **Leer** cada carácter con read(). Devuelve **-1** cuando ha llegado al final

```
int c = fr.read();  
while (c != -1) {  
    System.out.print((char) c);  
    c = fr.read();  
}
```

4. **Cerrar** el flujo

```
fr.close();
```

RESUMEN

ESCRITURA DE FICHEROS DE TEXTO SIN BUFFER (CARÁCTER A CARÁCTER)

1. Crear un **objeto File** con el fichero

```
File fichero = new File ("fichero.txt");
```

2. Crear el **objeto FileWriter**

```
FileWriter fw = new FileWriter(fichero);
```

3. **Escribir** cada carácter con write()

```
fw.write('A');
```

4. **Cerrar** el flujo. Si no los datos no se escriben en el fichero

```
fw.close();
```


RESUMEN

LECTURA DE FICHEROS DE TEXTO CON BUFFER (LÍNEA A LÍNEA)

1. Crear un **objeto File** con el fichero
`File fichero = new File ("fichero.txt");`
2. Crear el **objeto FileReader**
`FileReader fr = new FileReader(fichero);`
3. Crear el objeto **BufferedReader** partiendo del FileReader anterior
`BufferedReader br= new BufferedReader(fr);`
4. **Leer** cada línea con `readLine()`. Devuelve **null** cuando ha llegado al final

```
while((cadena=br.readLine())!=null) {  
    System.out.println(cadena);  
}
```
5. **Cerrar** el flujo
`br.close();`

RESUMEN

ESCRITURA EN FICHEROS DE TEXTO CON BUFFER

1. Crear un **objeto File** con el fichero
`File fichero = new File ("fichero.txt");`
2. Crear el **objeto FileWriter**
`FileWriter fw = new FileWriter(fichero);`
3. Crear el objeto **BufferedWriter** partiendo del FileWriter anterior
`BufferedWriter bw= new BufferedWriter(fw);`
4. **Escribir** en el fichero con `write()`
`bw.write("Escribiendo con buffer");`
5. **Cerrar** el flujo. Si no los datos no se escriben en el fichero
`bw.close();`

RESUMEN

ESCRITURA EN FICHEROS DE TEXTO CON PRINTWRITER (BUFFER)

1. Crear un **objeto File** con el fichero
`File fichero = new File ("fichero.txt");`
2. Crear el **objeto FileWriter**
`FileWriter fw = new FileWriter(fichero);`
3. Crear el objeto **PrintWriter** partiendo del FileWriter anterior
`PrintWriter pw= new PrintWriter(fw);`
4. **Escribir** en el fichero con `println()`
`pw.println("Escribiendo con print writer");`
5. **Cerrar** el flujo. Si no los datos no se escriben en el fichero
`pw.close();`

RESUMEN

LECTURA DE FICHEROS BINARIOS CON INPUTSTREAM

1. Crear un **objeto File** con el fichero

```
File fichero = new File ("fichero.dat");
```

2. Crear el **objeto FileInputStream**

```
FileInputStream fis = new FileInputStream(fichero);
```

3. **Leer** el fichero con read(). Devuelve -1 cuando ha llegado al final

```
int num = fis.read();  
while (num != -1) {  
    System.out.print(num);  
    num = fis.read();  
}
```

4. **Cerrar** el flujo

```
fis.close();
```

RESUMEN

ESCRITURA EN FICHEROS BINARIOS CON OUTPUTSTREAM

1. Crear un **objeto File** con el fichero

```
File fichero = new File ("fichero.dat");
```

2. Crear el **objeto FileOutputStream**

```
FileOutputStream fos = new FileOutputStream(fichero);
```

3. **Escribir** en el fichero con write()

```
fos.write(34);
```

4. **Cerrar** el flujo. Si no los datos no se escriben en el fichero

```
fos.close();
```

RESUMEN

SERIALIZACIÓN. LECTURA

1. Crear un **objeto File** con el fichero
`File fichero = new File ("fichero.dat");`
2. Crear el **objeto FileInputStream**
`FileInputStream fis = new FileInputStream(fichero);`
3. Crear el **objeto ObjectInputStream**
`ObjectInputStream ois=new ObjectInputStream(fis);`
4. **Leer** del fichero con `readObject()`. Hacer cast
`while((p=(Persona)ois.readObject())!=null) {
 System.out.println(p.toString());}`
Tratar la excepción EOFException
5. **Cerrar** el flujo
`ois.close();`

RESUMEN

SERIALIZACIÓN. ESCRITURA

1. Crear un **objeto File** con el fichero

```
File fichero = new File ("fichero.dat");
```

2. Crear el **objeto FileOutputStream**

```
FileOutputStream fos = new FileOutputStream(fichero);
```

3. Crear el **objeto ObjectOutputStream**

```
ObjectOutputStream oos=new ObjectOutputStream(fos);
```

4. **Escribir** en el fichero con writeObject(). **IMPORTANTE:** La clase debe ser **serializable**

```
oos.writeObject(new Persona("Pepe", 25));
```

5. **Cerrar** el flujo. **Si no los datos no se escriben en el fichero**

```
oos.close();
```

EJERCICIOS

[Soluciones a los ejercicios de las presentaciones](#)

FIN!